4.1构造函数

- 构造函数是类的成员函数，可以在类内定义，可可以在类外定义；
- 若定义的类要说明该类的对象时，构造函数必须是公有的成员函数。如果定义的类仅用于派生其它类时，则可将构造函数定义为保护的成员函数。
- 由于构造函数属于类的成员函数，它对私有数据成员、保护的数据成员和公有的数据成员均能进行初始化。

4.1构造函数

- 一个类可以定义若干个构造函数。当定义多个构造函数时，必须满足函数重载的原则。
- 构造函数可以指定参数的缺省值。
- 每一个对象都必须有相应的构造函数
- 在定义类时，若没有定义类的构造函数，则编译器自动产生一个缺省参数的构造函数
- 只要显式定义了一个类的构造函数，则编译器就不产生缺省参数的构造函数

4.1构造函数

构造函数可重载，可设置缺省参数，每个对象均有对应构造函数

```
class  Point{
        float   x,y;
public:
        Point(float a, float b=10)     {  x=a;   y=b;  }
        Point(){    x=0;  y=0;}
        void  Print() {cout<<x<<'\t'<<y<<endl;  }
};
int main()
{     Point  a1, a2(20.0), a3(3.0,  7.0);
      a1.Print();
      a2.Print();
      a3.Print();
}
```

带缺省参数的构造函数

不带参数的构造函数

```
0       0
20      10
3       7
```

每一个对象必须要有相应的构造函数

4.1构造函数

定义类时，若没有构造函数，则编译器自动产生一个缺省的构造函数，缺省的构造函数不对所产生对象的数据成员赋初值；

```
class Point{
        float  x,y;
public:

   void Set(float a,float b) {    x=a;   y=b;}
   void Print()    {     cout<<"x="<<x<<'\t'<<"y="<<y<<endl;   }
};
int main()
{       Point a1,a2;
        a1.Set (2.0,4.0);
        a1.Print ();
        a2.Print();
 }
```

缺省的构造函数，编译器自动产生,可以不写

产生对象时，自动调用缺省的构造函数，不赋值

4.1构造函数

在定义类时，只要显式定义了一个类的构造函数，则编译器就不产生缺省的构造函数

```
class Point{
        float x,y;
public:
   Point(float a,float b) {x=a;        y=b;    }
  void Print()
   {     cout<<x<<'\t'<<y<<endl;     }
};
int main()
{       Point  a1;
        Point a2(3.0,30.0);
      return 0; }
```

显式定义了构造函数，不产生缺省的构造函数

error,定义时，没有构造函数可供调用

4.1构造函数

◆ 在类中，若定义了没有参数的构造函数，或各参数均有缺省值的构造函数也称为缺省的构造函数，**缺省的构造函数只能有一个。**

```
class Point{
        float x,y;
public:
   Point(float a=0,float b=0)
   {     x=a;    y=b;    }
   Point(){a=1;b=1; }
  void Print()
   {     cout<<x<<'\t'<<y<<endl;     }
};
int main()
{       Point  a1;
        Point a2(3.0,30.0);
      return 0; }
```

两个函数均为缺省的构造函数

产生对象时，系统必定要调用构造函数。所以任一对象的构造函数必须唯一。

Error! 两个构造函数均可供调用，构造函数不唯一

4.2 析构函数

- 析构函数的作用与构造函数正好相反，是在对象的生命期结束时，释放系统为对象所分配的空间，即要撤消一个对象。
- 析构函数是类的成员函数，函数体可写在类体内，也可写在类体外。
- 析构函数是一个特殊的成员函数，函数名必须与类名相同，并在其前面加上字符“~”，以便和构造函数名相区别。
- 析构函数不能带有任何参数，不能有返回值，不指定函数类型。

4.2 析构函数

- 一个类中，只能定义一个析构函数，**析构函数不允许重载。**
- 析构函数是在撤消对象时由系统自动调用的。
- 在程序的执行过程中，当遇到**某一对象的生存期结束时，系统自动调用析构函数**，然后再收回为对象分配的存储空间。
- 如果程序中未声明析构函数，编译器将自动产生一个缺省的析构函数。

4.2 析构函数

```
class Point{
        float x,y;
public:
  Point(float a,float b)
   {  x=a;y=b;cout<<"调用非缺省的构造函数\n";}
  Point()
   {  x=0;  y=0;  cout<<"调用缺省的构造函数\n" ;}
  ~Point() {    cout<<"调用析构函数\n";}
   void Print() {    cout<<x<<'\t'<<y<<endl;}
};
int main( )
{       Point  a1;
        Point  a2(3.0,30.0);
        cout<<"退出主函数\n";
      return 0;  }
```

```
调用缺省的构造函数
调用非缺省的构造函数
退出主函数
调用析构函数
调用析构函数
```

程序改错

```
class rectangle
{
      int x1,y1,x2,y2;
public:
          Rectangle();
          rectangle(int a,int b,int c,int d);
          void change(int,int,int,int) ;
          void display();

}
```



```
int main()
{
          Rectangle p1();
           p1.x1=p1.y1=2;
           p1.x2=p1.y2=4;
           rectangle p2(0,0,2,2);
            p2.display;
           return 0;
}
```

思考讨论



局部对象、静态局部对象、全局对象的构造和析构函数什么时候调用？

构造函数示例2

```
class  Point
{
    float   x,y;
 public:
   Point(float a, float b){x=a;y=b;cout<<"初始化自动局部对象\n";}
   Point (){  x=0;  y=0;  cout<<"初始化静态局部对象\n";}
   Point (float  a){ x=a;  y=0;  cout<<"初始化全局对象\n"; }
  ~Point (){  cout<<“调用析构函数”  <<endl;  }
};
Point  a0(100.0);
void f(int x, int y)
{  cout<<" 进入f()函数\n";
    Point ab(x,  y）；
    static  Point a3;
}
int main()
{    cout<<"进入main函数\n";      f(10,20);
     f(20,40); cout<<“main函数结束\n"; return 0;
}
```

全局对象

局部对象

静态局部对象

初始化全局对象
进入main函数
进入f()函数
初始化自动局部对象
初始化静态局部对象
调用析构函数
进入f()函数
初始化自动局部对象
调用析构函数
main函数结束
调用析构函数
调用析构函数

不同类型的对象构造函数及析构函数的调用

全局对象

在程序开始执行时，调用构造函数；
到程序结束时，调用析构函数。

局部对象

当程序执行到定义对象的地方时，调用构造函数；
在退出对象的作用域时，调用析构函数。

静态局部对象

在首次到达对象的定义时调用构造函数；
到程序结束时，调用析构函数

5、复制构造函数

5.1、复制构造函数

同类型的对象在内存中有完全相同的结构，可以相互赋值，相当于类中的数据成员相互赋值；

可以使用一个对象（参数指定的对象），去初始化一个正在被建立的同类型对象，此时系统需要调用**复制构造函数**。

5、复制构造函数

复制构造函数是一种特殊的构造函数，其形参为本类的对象引用。

```
class 类名
{ public：
      类名（形参）；
      类名（类名 &对象名）；
      …
};
```

构造函数

复制构造函数声明

类外复制构造函数定义

```
类名:: 类名（类名 &对象名）
 {    函数体    //完成对应数据成员的赋值}
```

5、复制构造函数


```
class Point
{
  public:
     Point(int a=0,int b=0){X=a; Y=b;}
     Point(Point&  p);
     int GetX() {return X;}
     int GetY() {return Y;}
  private:
     int  X,Y;
};
```

复制构造函数的声明
形参为本类的对象引用

```
Point::Point (Point&  p)
{
     X=p.X;
     Y=p.Y;
     cout<<"拷贝构造函数被调用"<<endl;
}
```

参数必须是同类型对象的引用

5、复制构造函数应用举例

- **(1) 当用类的一个对象去初始化该类的另一个对象时系统自动调用它实现拷贝赋值。**

```
int main()
{  Point A(1,2);
   Point B(A);
   cout<<B.GetX( )<<endl;
    return 0;
}
```

复制构造函数被调用

5、复制构造函数应用举例

- **(2)若函数的形参为类对象，调用函数时，实参赋值给形参，系统自动调用拷贝构造函数。**

```
void fun1(Point p)
{   cout<<p.GetX（ ）<<endl;
}
int main（ ）
{   Point A(1,2);
    fun1(A);
    return 0;
}
```

形参是对象

实参是对象

调用函数时，内存中新建一个局部对象，复制构造函数被自动调用，把实参值复制到新的对象中

5、复制构造函数应用举例

- **(3)当函数的返回值是类对象时，系统自动调用拷贝构造函数。**

```
Point fun2( )
{   Point A(1,2);
    return A;
}
int main( )
{
     fun2( );
    return 0;
}
```

函数返回值是类对象，函数执行结束返回调用者时，复制构造函数被自动调用

拷贝构造函数示例

5、复制构造函数

如果没有定义完成拷贝功能的构造函数，编译器自动生成一个隐含的完成拷贝功能的构造函数，依次完成类中对应数据成员的拷贝。

```
Point::Point(Point &p)
{
    x=p.x;
    y=p.y;
}
```

隐含的构造函数

5、复制构造函数


```
class  Point
{
   float   x,y;
   public:
  Point(float a=0, float b=0)
  {
        x=a;  y=b;
        cout<<"调用了构造函数\n";
  }
  void  Print()
   {  cout<<x<<'\t'<<y<<endl;  }
  ~Point()
    {  cout<<"调用了析构函数\n";  }
};
```

隐含了拷贝的构造函数

```
int main()
{
     Point   a1(1.0,2.0);
     Point   a2(a1);
     Point   a3=a1;
     a1.Print();
     a2.Print();
     a3.Print();
      return 0;
}
```

```
调用了构造函数
1        2
1        2
1        2
调用了析构函数
调用了析构函数
调用了析构函数
```

成员对象和构造函数

类中的成员也可以是成员对象---使用其他类的对象作为类的成员。

```
class Line
{
    private:
        Point  p1,p2; //线段的两个端点
    public:
        Line(Point a,Point b); //构造函数
        void Draw( ); //画出线段
};
```

在类Line中包含类Point的对象

对类Line的对象初始化的同时还要对其成员数据类Point的对象进行初始化，所以，类Line的构造函数中要调用类Point的构造函数。

成员对象和构造函数

含有对象成员的类的构造函数格式如下：

```
类名::构造函数名(参数表):对象成员1(参数表1),..,对象成员(参数n)
{
    ..... //对其它成员的初始化
}
```

- 当建立类对象时：

  先调用各个对象成员的构造函数，初始化相应对象成员；

  然后才执行类的构造函数，初始化类中的其它成员。

- 撤销对象时：析构函数的调用顺序与构造函数正好相反。

- 对对象成员的构造函数的调用顺序取决于这些对象成员在类中说明的顺序，与它们在成员初始化列表中的顺序无关。

含有成员对象的类的构造函数举例1

```
class A{
        float x;
  public:
      A(int a) { x=a; cout<<"调用A的构造函数\n";}
      ~A(){ cout <<"调用A的析构函数\n";}
};
class B{
        float y;
  public:
      B(int a) { y=a; cout<<"调用B的构造函数\n";}
      ~B(){ cout <<"调用B的析构函数\n";}
};
class C{
        float z;
        B b1;
        A a1;
  public:
     C(int a,int b,int c):a1(a+b) ,b1(b)
       {z=c; cout<<"调用C的构造函数\n" }
     ~C(){cout <<"调用C的析构函数\n";}
};
int main()
{
    C c1(1, 2, 3);
  return 0;
}
```

A的构造函数

A的析构函数

B的构造函数

B的析构函数

对象成员

对象初始化,参数可以是表达式

C的构造函数

C的析构函数

```
调用了B的构造函数
调用了A的构造函数
调用了C的构造函数
调用了C的析构函数
调用了A的析构函数
调用了B的析构函数
```

对象的创建和撤销举例2

```
class complex
{
      double real, imag;
  public:
    complex(){  real=0.0;        imag=0.0;
                    cout<<“初始化  0   0"<<endl;       }
    complex(double r,double i=0.0)
     {     real=r;            imag=i;
        cout<<“初始化"<<r<<'\t'<<i<<endl;          }
    complex(complex &com);
    ~complex(){ cout <<“析构" "<<real<<'\t'<<imag<<endl;     }
     void assign(complex com)
              {real=com.real+1; imag=com.imag+1;   }
    void print(){cout<<real<<'+'<<imag<<'i'<<endl;}
};
```

对象的创建和撤销举例2

```
complex::complex(complex &com)
{
    cout<<“拷贝”<<com.real;
     cout<<'\t'<<com.image<<endl;
     real=com.real;
    image=com.image;
}
```

复制构造函数

```
complex fun(complex com)
{
    cout<<“进入fun函数"<<endl;
    global.assign(com);
    cout<<“离开fun函数"<<endl;
    return global;
}
```

函数，形参和返回值是类对象

```
complex global;
int main()
{
    cout <<“进入主函数"<< endl;
    complex com1, com2(5.6, 7.5);
    complex com3=com1;
    global.print();
    com1=fun(com2);
    com1.print();
    cout<<“离开主函数"<<endl;
   return 0;
}
```

全局对象

对象的创建和撤销举例2

初始化 0 0

进入main函数

初始化 0 0

初始化 5.6 7.5

拷贝 0 0

0+ 0i

拷贝 5.6 7.5

进入fun函数

拷贝 5.6 7.5

析构 5.6 7.5

离开fun函数

拷贝 6.6 8.5

析构 5.6 7.5

析构6.6 8.5

6.6+8.5i

离开主函数

析构 0 0

析构 5.6 7.5

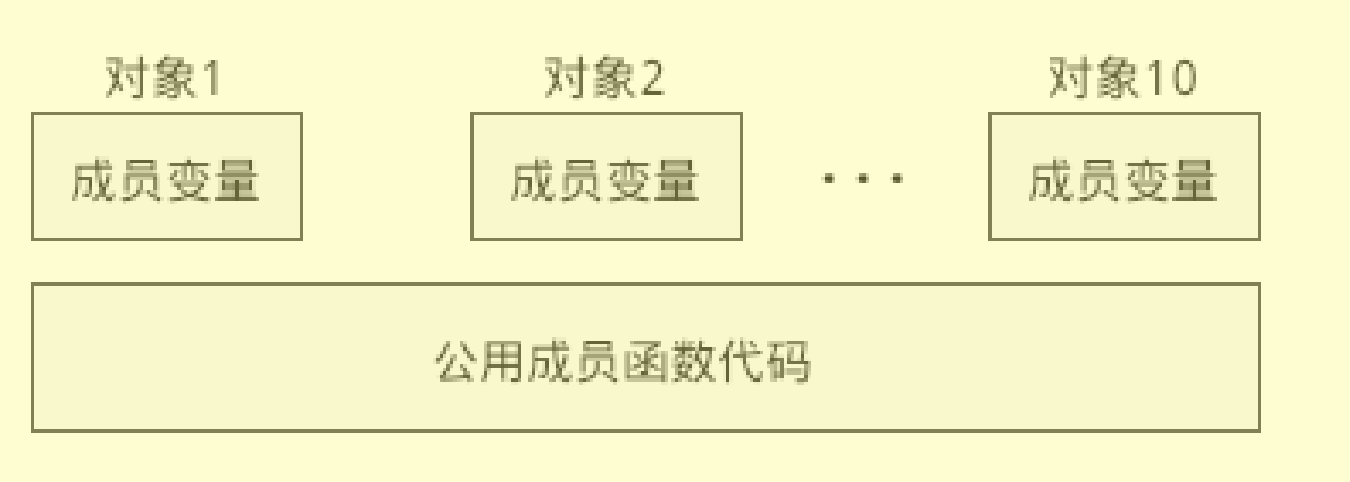
析构 6.6 8.5

析构6.6 8.5

6、this指针

6、this指针

用同一个类创建很多个对象，它们共用同一份成员函数的拷贝。成员函数如何判断该取哪一个对象的成员数据？



6、this指针

调用一个成员函数时，系统自动产生一个隐藏的指针（this指针），它始终指向产生这个调用的对象，并将该指针作为一个参数自动传递给该成员函数。

在 C++ 中，每一个对象都能通过 this 指针来访问自己的地址。this 指针是所有成员函数的隐含参数。

6、this指针


```
class Box
{
 public:
      Box(int h,int w,int len): height(h), width(w), length(len) { }
      int volume( );
 private:
       int height;
       int width;
       int length;
};
int Box::volume( )
{
     return   height*width*length;
}
```

编译系统:
volume(Box * this)

编译系统:
volume(&a[0])

```
int main( )
{
   Box a[3]= {   Box(10,12,15),
                 Box(15,18,20),
                 Box(16,20,26)
             };
    cout<<a[0].volume( )<<endl;
    cout<<a[1].volume( )<<endl;
    cout<<a[2].volume( )<<endl;
     return 0;
}
```

编译系统:
this->height*this->width*this->length

6、this指针

主程序中定义的Box类，

定义了3个同类对象a[3]，

三个函数调用：

```
a[0].volume();
a[1].volume();
a[2].volume();
```

调用成员函数a[0].volume()时，编译系统就把对象a[0]的起始地址赋给this指针，在成员函数引用数据成员时，就按照this的指向找到对象a[0]的数据成员。

volume函数要计算height*width*length的值，

实际上是执行：

```
(this->height)*(this->width)*(this->length)
```

由于当前this指向a，因此相当于执行：

```
(a[0].height)*(a[0].width)*(a[0].length)
```

6、this指针(例)


```
class A
{
    int i;
  public:
    int get() {return i;}
    void set(int i);
};
void A::set(int x)
{
    i=x;
    cout<<“this指针保存的内存地址为:”;
    cout<<this<<endl;
}
```

void set(A *this, int x)

this指针通常是隐式的

6、this指针(例)


```
对象a所在的内存地址为:002FFEE0
this指针保存的内存地址为:002FFEE0
对象a所保存的值为:9

对象b所在的内存地址为:002FFED4
this指针保存的内存地址为:002FFED4
对象b所保存的值为:999
请按任意键继续. . .
```

```
int main()
{
    A a, b;
    a.set(9);
   cout<<"对象a所在的内存地址为:"<<&a<<endl;
   cout<<"对象a所保存的值为:"<<a.get()<<endl;
   cout<<endl;
   b.set(999);
   cout<<"对象b所在的内存地址为:"<<&b<<endl;
   cout<<"对象b所保存的值为:"<<b.get()<<endl;
   return 0;
}
```

set(&a,9)

set(&b,999)

6、this指针

this指针通常是隐式的，但有时必须写成显式的，如在对复数类的赋值号重载中需要返回当前调用的对象时：

```
Complex & Complex::myassign (Complex& c)
{
        real = c.real;
        image = c.image;
         return *this;
}
```